Tashfeen, Ahmad
CSCI 6233: Machine Learning

Homework 5

Est genus in totidem tenui ratione redactum
scriptula, quot menses lubricus annus habet;
Parua tabella capit ternos utrimque lapillos,
in qua uicisse est continuasse suos.

There is another game divided into as many parts
as there are months in the year;
A small board has three pieces on either side,
the winner must get all the pieces in a straight line.
—Ovid (Ars Amatoria III, lines 365-369)

Question 1. Implement the Minimax algorithm for adversarial search in the Tic-tac-toe game tree.

Start by downloading and reading the starter code. You need to only edit the `model.py` by implementing the unfinished Python functions. Upon successfully implementing all of the API, you may run `python3 view.py`. You'll be presented to make the first move. Click at the appropriate place to start the game.

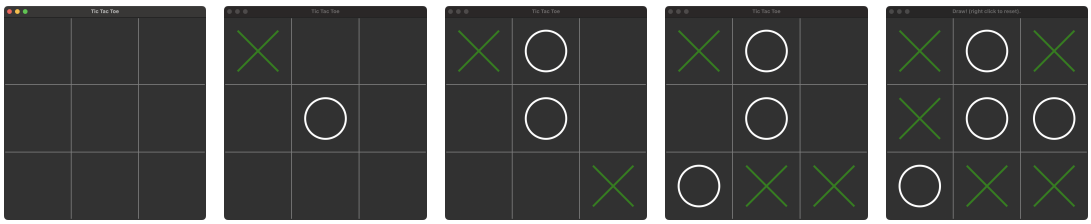

FIGURE 1. Tic-tac-toe gameplay in state of the art graphics by Tashfeen Studios™.

Question 2. How many unique terminal states are possible for the three by three Tic-tac-toe game?

Question 3. Once your game is functional and your `model.py` class contains a game tree. You should run `python3 model.py` to run the model's main code,

```
if __name__ == '__main__':
    x = Model()

    def dfs(root, leaves, player):
        """
        Implement depth first search to count the leaves for where min/-1,
        max/1 or draw/0, i. e., "player" occurs.
        """
        pass
```

Implement the `dfs(root, leaves, player)` as described in the doc-string to answer the following questions,

1) How many total leaves does the game tree rooted at `model.root` have? Is this number greater or less than your answer in question 2? What does that tell you about the uniqueness of the leaves?
2) How many tree-leaves result in a draw? A win for the second (min) player? A win for the first (max) player?

SUBMISSION INSTRUCTIONS

1) Turn in a PDF containing any plots, figures and/or answers from the homework.
2) Turn in your `model.py`.